PROJECT REPORT · B.TECH MAJOR PROJECT

VJLM

The Vignana Jyothi Language Model — a local, role-aware assistant for VNR Vignana Jyothi Institute of Engineering & Technology, built end to end on commodity CPU hardware with no external language-model service at runtime.

Author Vaishnavi Sriyaa N

Repository github.com/nvaishnavisriyaa/VJLM

Period covered 16 September 2026 – 22 October 2026

Report date 22 October 2026

Contents

1 What this project is, and what it refuses to do 3

2 Results, up front 4

3 Timeline at a glance 6

4 Foundation, reconnaissance and classification — 16 September 7

5 The dynamic data pipeline — 16–17 September 9

6 Extraction, chunking and enrichment 12

7 Three things that were silently broken — 30 September 14

8 The benchmark problem, and the similarity measure — 26 September 17

9 Fine-tuning: five configurations — 18–19 September 20

10 Compression for CPU — 21 September 24

11 Serving: authentication, prompt, interface 27

12 October: the honest re-measurement 28

13 The three-signal retrieval metric 31

14 The WordNet question, answered with numbers 34

15 Where the system stands — 22 October 36

16 What is not done 38

17 What the project taught about method 40

Appendix A Full dated timeline 42

Appendix B Every rejected experiment, with the number that killed it 44

Appendix C Repository map 46

1 What this project is, and what it refuses to do

> VJLM answers questions about VNRVJIET — attendance rules, examination regulations, fees, hostel policy, clubs, the academic calendar — from the institute's own published documents, on a laptop, with no internet call to any language-model provider at the moment a student asks.

That last clause is the whole difficulty. It would be straightforward to point a hosted model at the college website and get good answers. Nearly every design decision in this report follows instead from a short list of things the project is not allowed to do, and the interesting parts are where those constraints forced a measurement that a looser project would never have made.

The constraints, and where they came from

No external language-model service at runtime. The assistant runs on the institute's own hardware. A student's question, and the passages retrieved to answer it, never leave the machine. This was set by the project owner on 16 September and never relaxed. A later revision that day permitted free third-party *services* during development — package registries, Colab for training — but explicitly not language-model APIs for extraction or inference.

CPU only, at serving time. The target machine has no usable GPU for inference. Training was allowed to happen elsewhere; answering is not. This is what makes compression a first-class phase rather than an afterthought, and it is why a 2.46 GB model that runs at 14.7 tokens per second is a better outcome than a 7 GB model that scores marginally higher.

The pipeline must maintain itself. A college website changes constantly: new circulars, revised calendars, moved PDFs, deleted pages. A retrieval system built on a one-off scrape is wrong within a month. The ingestion layer therefore detects new, changed, moved and deleted content and re-syncs, and it runs when the application starts rather than from a scheduler entry that somebody would eventually forget.

No retraining, from October onward. Once the model was trained and compressed, re-training became computationally out of reach. Every improvement after 21 September had to come from retrieval or from code, never from the model's weights. This constraint is more clarifying than it sounds: it meant every October measurement had to answer "can this be fixed without touching the model?", and most of the time the answer was yes.

The retrieval measure had to be the supervisor's. A graded term ladder — full credit if the word is present, half credit for a synonym, nothing otherwise — was proposed as the similarity measure for the project. It is implemented as specified, and the three-signal hybrid that ships is a strict extension of it: set the semantic weight to zero and you recover the original measure exactly. That mattered when the hybrid turned out to beat it, because the comparison is then between a measure and its own superset, not between two unrelated ideas.

What exists at the end of the period covered

A crawl of 3,329 catalogued sources producing 104,965 retrievable passages; a Llama-3.2-3B model fine-tuned on institutional question-answer pairs and compressed to 2.46 GB of GGUF with no

measurable quality loss; a retrieval stage that combines keyword, synonym and semantic matching and places the answer among the five passages shown to the model for 86% of a 98-question test set; and a decision log of roughly 1,600 lines recording, for every choice, the number that justified it — including the choices that turned out wrong.

What does not exist is a system that answers 95% of questions correctly. The honest figure is lower, and section 16 says how much lower and why.

2 Results, up front

> Four numbers carry the project: the corpus it reads, the model it runs, the compression that makes it fit, and the retrieval that decides what the model sees. Only the last is still moving.

The deployed system

Component	What ships	Measured
Corpus	3,297 documents → 104,965 passages	3,329 sources catalogued
Base model	Llama-3.2-3B-Instruct, LoRA r=16	grounded F1 0.7567
Compression	GGUF Q6_K with a VJLM imatrix	2.46 GB, KLD 0.0097
Inference	llama.cpp, CPU, 20 threads	14.70 tok/s, ~2.4 s/answer
Retrieval	Three-signal hybrid, 5 passages	Hit@5 86%
Retrieval cost	BM25 ∪ dense pool of 1,200	~0.2 s/query

Fine-tuning: the five arms that decided the model

All five were scored on the same 147 held-out items, same seed, same decode settings, with 147 of 147 rows succeeding in every arm.

	Mistral-7B base	7B QLoRA	Llama-3B base	3B LoRA ★	3B QLoRA
grounded_f1	0.4401	0.7654	0.2369	**0.7567**	0.7567
abstain_recall	1.0000	1.0000	0.2833	**1.0000**	1.0000
false abstentions	14 / 87	0	12 / 87	**0**	0
citation rate	0.3605	0.5102	0.0204	0.5102	0.5170
training minutes	—	230.6	—	**74.0**	105.1
peak VRAM	—	7.18 GB	—	10.61 GB	4.88 GB
seconds / answer	22.19	20.16	5.83	13.72	13.24

Retrieval: what each signal is actually worth

Measured on 98 hand-written questions, reporting Hit@5 — the share of questions whose answer appears in the five passages the model is shown.

Configuration	Hit@5	Answer in candidate pool
Keyword matching alone	71%	98%
Keyword + synonym (the supervisor's coverage measure)	74%	99%
Keyword + semantic	82%	99%
All three signals — what ships	86%	99%
All three, weights normalised to sum to 1	80%	99%

> **The single most important line in this report.** The answer is present in the candidate pool for **99%** of questions but in the top five for **86%**. Nothing is unreachable; the remaining gap is entirely a question of ordering. That is a much more tractable problem than a missing-data problem, and it is also why no amount of additional crawling would help.

And the number that is not good yet

Hit@5 measures whether the answer was *retrieved*. It says nothing about whether the model then wrote it down correctly. The last end-to-end measurement of answer correctness over the same 98 questions was **57% by strict pattern match**; hand-grading all 42 failures found 7 that were correct answers scored wrong by the pattern, 4 genuinely ambiguous, and 31 actually wrong — so roughly **64% real**. That measurement predates the 22 October fixes and has not been repeated. It is the gap between "the system found the right page" and "the system gave the right answer", and it is where the next phase of work belongs.

3 Timeline at a glance

> Five weeks, in two distinct halves. The first three weeks built a working system quickly. The last two weeks were spent discovering that several of its measurements had been wrong, and fixing that.

Date	What happened	Phase
16 Sep	Repository foundation; no-LLM-API policy recorded; site reconnaissance; 14 website categories classified as fine-tune / RAG / both / neither; first working pipeline end to end; enrichment layer; Colab training harness	0–4
17 Sep	Train/test leakage found and fixed in the fine-tuning dataset; self-maintaining pipeline with app-driven refresh and headless page rendering; first dense index; stale-vector bug in the dense cache fixed; freshness gated on the site's deploy signal	3–4
18 Sep	Mistral-7B QLoRA trained and scored; Llama-3B LoRA scored — the 3B model matches the 7B; long runs made survivable; three earlier hardware and Colab decisions superseded	8
19 Sep	Llama-3B QLoRA exceeds 16 GB VRAM; two failed fixes recorded as failures; arm C finally trained; **LoRA vs QLoRA settled as a wash on quality**	8
21 Sep	Both models compressed to GGUF; the quantization ladder walked and gated on KL divergence; Q4_K_M built and rejected; adapters published	9
26 Sep	Eight similarity modes measured on 90 questions; the supervisor's coverage measure selected; unrestricted WordNet expansion shown to be noise	6
30 Sep	Three silent defects found: the corpus held almost no student-policy content; 25 of 672 PDFs had no text layer; dense vectors were misaligned with the lexical index	3–6
12 Oct	Dropped query terms recovered; the model made to answer rather than refuse when the answer was retrieved; coverage became the default retriever	11–12
17 Oct	Corpus rebuilt after the classifier fix: 104,965 passages from 3,297 documents, zero parse failures	4
18 Oct	Three-signal fusion built: the semantic arm promoted from re-ranker to first-stage retriever, pool deepened to 1,200, branch mismatch penalised. Six variants measured and rejected	11
22 Oct	A 14-point prompt-assembly leak found and fixed; WordNet re-implemented with an IDF gate; weights swept over 36 configurations; corpus audited and cleared; two chunker defects fixed	11–12

On the shape of that timeline. The work between 16 and 21 September looks fast because it was: a pipeline, five training runs and a full quantization ladder in six days. It was fast partly because the benchmark used to judge retrieval during that period was generating its own questions from the passages it was testing, which flattered everything. The October weeks are slower because they are mostly the cost of that shortcut, paid late.

4 Foundation, reconnaissance and classification

16 SEPTEMBER 2026

> Before any code: where does the data come from, what is it allowed to be used for, and which parts of it belong in a model's weights rather than in a search index?

4.1 The dependency policy

The first decision recorded in the log is not technical. It is a policy: the ingestion and preprocessing pipeline may not call a language-model API. The reason is that an extraction pipeline built on a hosted model is not reproducible by the institute, not auditable, and not free. Every field VJLM derives from a document — department, regulation year, academic year, document type — is therefore produced by deterministic code that can be read and corrected.

Later the same day the policy was revised once, and the revision is worth quoting in substance: free third-party services are permitted, language-model APIs are not. This is what allowed Colab to be used for training while keeping the data pipeline honest. It is a narrower rule than "no external services", and the narrowness is deliberate — a blanket ban would have made the project impossible on the available hardware without improving its integrity.

4.2 Site reconnaissance, and the subdomain that says no

The institute publishes across two hosts. The main site, `www.vnrvjiet.ac.in`, is a modern single-page application: its content is assembled in the browser, so a naive HTTP fetch returns a shell with no text in it. The document host, `docs.vnrvjiet.ac.in`, carries the PDFs that matter most — regulations, syllabi, calendars, circulars — and its `robots.txt` disallows all crawlers.

Both facts shaped the ingestion layer. The first forced headless page rendering rather than plain fetching, which is slower and more fragile but is the only way to see what a student sees. The second required an explicit, recorded decision about how that material is obtained, rather than a crawler that quietly ignores the directive.

4.3 Fine-tune, retrieve, both, or neither

The fourteen content categories on the public website were classified against a single question: does this content change, and does its exact wording matter?

Treatment	Reasoning	Examples
RAG	Changes on a schedule, and the exact figure or date is the answer. Must never be remembered, only looked up.	Academic calendars, fee notifications, circulars, examination schedules
Fine-tune	Stable, and what matters is the manner of answering rather than a specific fact	Tone, role awareness, citation habit, when to refuse
Both	Stable structure, volatile specifics	Regulations — the clause structure persists, the numbers change by batch
Neither	Not useful to a student, or not ours to serve	Image galleries, recruitment notices, third-party embeds

This classification is the reason the project has a training set at all. A model fine-tuned on fee amounts would confidently recite last year's fees forever; a model fine-tuned on the *habit* of citing a source and refusing when the context does not contain the answer improves every answer the retrieval layer supports. The fine-tuning set was therefore built to teach behaviour, and the facts were left to retrieval.

> **Why this matters more than it appears.** The single largest measured behavioural improvement in the project comes from this classification, not from any retrieval work. Untuned Llama-3B answers 43 of the 60 questions it is supposed to refuse — it invents regulations. After fine-tuning on behaviour rather than facts, all three tuned arms refuse all 60, and lose every *false* refusal at the same time. That is the difference between a system that can be shown to a student and one that cannot.

5 The dynamic data pipeline

16–17 SEPTEMBER 2026

> The requirement was not "fetch the website". It was: detect what is new, what has changed, what has moved and what has been deleted, and keep the local knowledge base in step — without a human and without a scheduler.

5.1 The five stages

Stage	Module	Responsibility
Discover	`ingestion/crawler/discover.py`	Enumerate routes and linked documents; maintain the source registry
Fetch	`ingestion/crawler/fetch.py`	Conditional retrieval; per-source sync state; decide what "changed" means
Render	`ingestion/crawler/render.py`	Headless browser rendering for the single-page application
Extract	`preprocessing/extract.py`	PDF, HTML and DOCX to text; OCR fallback; chunking
Enrich	`preprocessing/enrich.py`	Metadata extraction, PII redaction, duplicate marking

5.2 What "automatic" turned out to require

The honest version of this story is that "the pipeline refreshes itself" took three attempts, and the first two were wrong in instructive ways.

The first approach compared content hashes. If a page's bytes changed, the page changed. This fails on a single-page application because the rendered output includes build identifiers, timestamps and hydration artifacts that change on every deploy whether or not any content did. Every source appeared to change every time, so the pipeline re-extracted the entire corpus on every run — correct, but useless.

The second approach gated on the site's own deploy signal: if the application bundle has not changed, nothing has changed. This is cheap and mostly right, and it is still in the system. But it is a site-wide signal, so it cannot say *which* page changed, and it says nothing about the PDFs, which live on a different host and are not part of any bundle.

What runs now combines them: the deploy signal decides whether to walk the site at all, per-source state decides which sources to re-fetch, and documents are tracked independently of the pages that link to them. A floor was added so that even with no signal at all the pipeline still re-checks periodically, because a signal that silently stops firing is worse than no signal.

> **A bug that produced correct-looking wrong answers.** The dense embedding cache was keyed in a way that let a stale vector survive a content change: the passage text was re-extracted and re-indexed, but its embedding was served from cache and still described the *old* text. Nothing failed. Retrieval returned confident, well-formed, wrong passages. This class of defect — silent, plausible, invisible to any test that only checks that results come back — recurs three more times in this report, and is the main methodological theme of the whole project.

5.3 Boilerplate, and why it is not hard-coded

The institute's site carries an announcement ticker — EAPCET code, NAAC grade, admission notices — that sits outside the navigation element and therefore survives structural stripping. It appeared on all 105 rendered pages. Removing it by matching its strings would work until the next redesign, at which point it would silently stop working.

Instead the extractor counts how many distinct pages each line of text appears on, and treats lines recurring across many pages as site chrome. This finds the ticker without naming it, and will find whatever replaces it. It is a small thing, but it is the pattern the whole pipeline follows: prefer a rule that derives the answer from the data over a rule that encodes today's answer.

5.4 Privacy, handled at the right layer

Staff emails and phone numbers appear throughout the corpus, in directories and circulars. They are not removed from the stored passages, because removing them would change the text the embeddings were computed over and desynchronise the vectors from the metadata. Instead redaction happens at prompt-assembly time: the model never sees an email address or a phone number, even though the index contains them. The index needs them to match a query about a department contact; the model does not need them to answer it.

6 Extraction, chunking and enrichment

> 104,965 passages from 3,297 documents, with zero parse failures on the final run. The interesting decisions are about how a document becomes passages, because that choice turns out to bound retrieval quality.

6.1 Chunking, and the cap that matters

Text is split on paragraph boundaries and packed up to 1,100 characters, with 150 characters of overlap and a 200-character minimum. Packing means a single passage may contain several consecutive paragraphs if they fit.

That packing decision is, in hindsight, the most consequential parameter in the retrieval system, and section 16 argues it is now the main thing holding accuracy down. The reason is dilution: all three retrieval signals score a *passage*, and if the answer is one sentence inside 1,100 characters, it competes for that passage's score against roughly 900 characters of unrelated text. A passage that mentions the right topic four times can outscore the passage that actually answers the question.

6.2 Two defects in the chunker, found 22 October

Long paragraphs — and PDF extraction produces many, since a whole regulation clause or an entire table often arrives as one paragraph — took a separate code path that sliced them at fixed character offsets:

```
if len(p) > max_c:
    for i in range(0, len(p), max_c - ov):
        out.append(p[i:i + max_c])
```

The original long-paragraph path, in `preprocessing/extract.py`.

Two things are wrong with it. It cuts at an arbitrary character, so passages begin and end mid-sentence and sometimes mid-word, which damages both the embedding and the model's ability to read the passage. And its final slice is frequently shorter than the 200-character minimum, at which point the filter at the end of the function discards it — so a 2,050-character paragraph silently lost its last 150 characters. Verified on exactly that case: three pieces produced, two kept.

Both are fixed. Cuts now land on the last sentence boundary before the limit, falling back to a word boundary, and an undersized tail is merged into the piece before it rather than dropped. The fix affects future extractions; it does not change the current corpus until the pipeline runs again.

6.3 Enrichment that does not rewrite text

Department, regulation, academic year and document type are added as metadata *beside* each passage rather than edited into it. The reasoning is recorded in the module itself: embeddings are computed over the original text, so mutating the text in place would silently desynchronise the vectors from the metadata — the same failure class as the stale-vector bug. Metadata can therefore be re-derived and corrected at any time without re-embedding 105,000 passages, which during October was done repeatedly.

6.4 Was the knowledge base ever the problem?

This question was put to a direct test on 22 October, because it would be easy to spend weeks on retrieval while the real fault was bad data. For each of the 17 questions retrieval was failing, the entire 104,965-passage corpus was searched for a passage containing the answer, and each hit was classified.

Verdict	Meaning	Count
Absent	No passage anywhere contains the answer — the crawl or extractor lost it	0 of 17
Blob	Present, but in one undivided page-sized passage	0 of 17
Table	Present but flattened into unreadable text	0 of 17
Readable	**Present in clean, correctly extracted text**	17 of 17

Four cases were initially flagged as mangled tables by an automatic shape heuristic. Dumped in full and read by eye, all four were legible: the clubs list has one club per line, the accreditation list reads `NAAC Accredited with 'A++' Grade NBA Accredited CE, EEE, ME...`, and the academic calendar contains `Commencement of Classes | 10 August 2026` — which is precisely the answer to the question that was failing. The heuristic was over-firing, and so was a parallel check for words glued together by PDF extraction: it flagged 3.6% of passages, but the hits were mostly real words such as `HISTOPATHOLOGICAL` and surnames. Genuine gluing is confined to PDF display headers, never body text.

> **Conclusion, stated plainly because it redirects everything after it:** the knowledge base is not the limiting factor. The answers are present and readable. One narrow extraction defect does remain — OCR misreads superscript ordinals in scanned PDFs, rendering `06th January` as `06m January`, which a human reads past but a keyword match cannot — and it is worth fixing, but it is not worth 14 questions.

7 Three things that were silently broken

30 SEPTEMBER 2026

> Two weeks after the system first worked end to end, three defects were found on the same day. None of them produced an error. All three invalidated measurements that had already been recorded as decisions.

7.1 The corpus contained almost no student-policy content

VJLM is meant to answer student questions about rules. A direct audit of 12,978 passages found that the corpus barely contained any. What it mostly contained was syllabi, committee minutes, audited balance sheets, placement statistics and departmental reports — everything the institute publishes, weighted by how much of it there is, rather than weighted by what a student would ask about.

The root cause was already recorded, which is the uncomfortable part: the source classifier was assigning categories in a way that under-collected the regulation and policy documents. It had been written on 16 September and noted as approximate. Its approximation turned out to be load-bearing.

The consequence for everything measured before that date is sharp. Retrieval quality had been measured on a corpus that largely did not contain the answers to the questions a student asks, which means the measurements were describing a different system from the one intended. The classifier was fixed and the corpus rebuilt — the rebuild that eventually produced the 104,965 passages reported here, finishing 17 October.

7.2 25 of 672 PDFs had no text layer at all

A browser inspection separated the document collection into categories that the extractor had been treating identically. Twenty-five PDFs are scans: images of pages, with no extractable text. The extractor processed them without complaint and produced nothing, so they were simply absent from the index while appearing in every count of successfully processed sources.

OCR was added for exactly that category. The recogniser then introduced a second defect of its own: it dropped spaces between English words, so `Commencement of Classes` became a single token. That had to be detected and repaired separately. The repair is recorded with its reasoning, because the obvious fix — inserting spaces wherever a dictionary word boundary appears — would corrupt legitimate compound identifiers such as regulation codes.

> **The category that lied.** The decision log's own heading for this entry is "OCR for image-only PDFs, and the category that lied". The lie was not the PDFs; it was the pipeline's success metric. "672 of 672 PDFs processed" was true and meaningless. A count of documents processed is not a count of documents *read*, and nothing in the system distinguished them until somebody looked at the files by hand.

7.3 Dense vectors silently misaligned with the lexical index

The third defect is the most serious. The dense vector matrix and the BM25 index are two parallel structures over the same passages, and retrieval assumes row i of one corresponds to entry i of the other.

A rebuild sequence existed in which that correspondence broke.

When it broke, nothing failed. Semantic retrieval returned passages — just the wrong ones, consistently, with plausible scores. Every retrieval measurement that involved the dense arm before the fix is therefore uninterpretable, including the comparison on 26 September that concluded cosine similarity was the weakest signal available.

That conclusion is now known to be wrong, and section 8 covers how it was reached and what replaced it. The misalignment is not the only reason it was wrong — the benchmark was also flawed — but it is sufficient on its own to invalidate the number.

7.4 What these three have in common

None of them raised an exception. Each one produced output that looked entirely normal: passages were returned, documents were counted as processed, measurements completed and were recorded with the care the decision log demands. The failure mode throughout this project has not been code that breaks; it has been code that works on the wrong thing while reporting success.

The practical response was to change what gets checked. It is not enough to verify that retrieval returns results, that extraction processed every file, or that a measurement ran to completion. The useful checks are the ones that ask whether the thing being measured was exercised at all — and as section 17 describes, that lesson had to be learned a second time in October, at the cost of six consecutive meaningless measurements.

8 The benchmark problem, and the similarity measure

26 SEPTEMBER 2026

> The project's supervisor proposed a specific similarity measure. It was implemented, compared against seven alternatives, and won decisively. The comparison was also, it later turned out, measuring the wrong thing.

8.1 The measure as proposed, and as implemented

The proposal is a graded term ladder: for each term of the query, award 1.0 if the word appears in the passage, 0.5 if a synonym appears, and 0 otherwise. The implementation adds one thing — IDF weighting — and the addition is not decoration:

$$\mathrm{Cov}(q, d) = \frac{\Sigma_t \, \mathrm{idf}(t) \cdot \mathrm{tier}(t, d)}{\Sigma_t \, \mathrm{idf}(t)} \in [0, 1]$$

The word `students` occurs in most of the corpus and separates nothing. The word `condonation` occurs in a handful of documents and separates almost perfectly. An unweighted average over query terms treats them as equally informative, which is plainly wrong for the questions students ask. Weighting each tier by the term's inverse document frequency means that covering a *discriminating* term counts for more than covering a common one.

One further detail: a query term that does not appear in the corpus at all is assigned the maximum observed IDF. It is maximally rare and unmatchable, so it only ever enters the denominator — correctly penalising every passage for failing to cover something distinctive, rather than being quietly ignored.

8.2 Eight modes, measured

Ninety benchmark questions, one index of 13,051 passages, same hardware:

Mode	Recall@5	MRR	p50 latency
coverage — the supervisor's ladder, IDF-weighted	0.944	0.836	35.0 ms
coverage + synonyms	0.944	0.836	14.0 ms
lexical (0.6·BM25 + 0.4·coverage)	0.933	0.822	17.4 ms
bm25 + synonyms	0.944	0.815	29.6 ms
bm25	0.911	0.803	19.0 ms
hybrid3 (0.35 / 0.25 / 0.40)	0.900	0.780	43.6 ms
hybrid (0.5 BM25 + 0.5 cosine)	0.789	0.722	33.5 ms
dense — cosine similarity alone	0.667	0.509	326.0 ms

Coverage won on both metrics and was among the cheapest. The explanation offered at the time is still correct as far as it goes: BM25 is a term-*frequency* measure, so a page repeating "attendance" fifty times outranks a page covering attendance *and* examinations *and* condonation. Coverage cannot be won by repetition, because a term contributes its IDF once or not at all, and breadth of coverage is the better signal for the multi-concept questions students actually ask.

8.3 Two predictions in the design document, both refuted

These are recorded because a decision log that preserves only correct guesses is worthless. The design document predicted that cosine similarity would be the strongest single signal, and assigned it the largest weight in the proposed hybrid. It finished *last*, with MRR 0.509 against coverage's 0.836, and seventeen times slower. The document also predicted that fusion would beat its parts; every blend tried lost to coverage alone, and lost in proportion to how much cosine it contained.

8.4 Why the whole table is unreliable

The ninety benchmark questions were generated automatically, by lifting roughly twelve words verbatim from the gold passage. That is not a paraphrase. It is a bag of exact keywords drawn from the very passage the system is supposed to find.

A benchmark of that construction structurally favours lexical matching and structurally penalises embeddings, because the one thing an embedding model offers — matching meaning across different wording — is exactly what the questions never require. The conclusion "cosine is the weakest signal" was therefore an artifact of the measuring instrument, compounded by the vector misalignment described in section 7.3.

> **What this cost.** From 26 September to 18 October the system ran with the semantic signal weighted at or near zero, on the evidence of this table. Rebuilt on hand-written questions in October, the same comparison shows semantic matching worth **+11 points** of Hit@5 — the largest single contribution of any signal. Three weeks of retrieval work were spent optimising a configuration that a flawed benchmark had recommended.

The lesson generalises past this project. The benchmark was built before the components were compared, which was the right order and is written into the project's phase gates. But building it first does not make it valid, and a benchmark that is cheap to generate is usually cheap because it has removed the difficulty the system exists to handle.

9 Fine-tuning: five configurations

18–19 SEPTEMBER 2026

> Two base models, two adaptation methods, and both bases untuned as controls. Scored on the same 147 held-out items with the same seed and decode settings, 147 of 147 rows succeeding in every arm.

9.1 What was being decided

Three questions at once, which is why five arms rather than two: does the 7B model justify its cost over the 3B model; does QLoRA's memory saving cost quality; and does fine-tuning actually matter against a well-prompted base model. The third is the one most projects skip, and it produced the largest effect.

9.2 The measurements

	Mistral-7B base	7B QLoRA	Llama-3B base	3B LoRA	3B QLoRA
grounded_f1	0.4401	0.7654	0.2369	0.7567	0.7567
abstain_recall	1.0000	1.0000	0.2833	1.0000	1.0000
false abstentions	14 / 87	0	12 / 87	0	0
citation_rate	0.3605	0.5102	0.0204	0.5102	0.5170
training minutes	—	230.6	—	74.0	105.1
peak VRAM	—	7.18 GB	—	10.61 GB	4.88 GB
seconds / answer	22.19	20.16	5.83	13.72	13.24

9.3 LoRA and QLoRA are behaviourally indistinguishable here

This was tested properly, as a controlled comparison: identical data, identical ordering, identical seed, schedule, rank, target modules, batch size and sequence length, with only the representation of the frozen base weights differing. The result is 0.756694 against 0.756702 — eight parts in a million, against a pre-registered noise floor of roughly 0.03.

It is worth stating why this is not simply a duplicated row, because that was the first suspicion. Sixty-seven of the 87 scored items receive *different* scores in the two arms; they average out. The two models behave differently item by item and identically in aggregate.

So the trade is not quality. It is VRAM against wall-clock: QLoRA used 54% less memory and took 42% longer, because the 4-bit NF4 base weights must be dequantized on every forward pass and again during the backward pass. For anyone choosing between them, that is the entire decision, and it depends only on which resource is scarce.

9.4 The 3B model matches the 7B once tuned

0.7567 against 0.7654, with confidence intervals overlapping across nearly their whole width — at a third of the training time and roughly half the inference time. Given a CPU serving constraint, this

settled the base model choice without needing a close call on quality.

9.5 Fine-tuning mattered far more than base-model choice

The two untuned bases differ by 0.2032 in grounded F1. The three tuned arms differ by 0.0087 across the board — a twenty-three-fold difference in effect size. Whatever distinguishes Mistral-7B from Llama-3B on general benchmarks is almost entirely erased by training on 518,000 tokens of institutional question-answer pairs.

The clearest single effect is in refusal behaviour. Untuned Llama-3B answers 43 of the 60 questions whose correct response is "I could not find this in the available VNRVJIET sources" — it invents regulations, dates and fees with complete fluency. After fine-tuning, all three tuned arms refuse all 60. And critically they do so while *losing* every false refusal: the base models each wrongly refuse 12 to 14 answerable questions, the tuned arms refuse none. Both error directions improved at once, which is unusual and is the strongest evidence in the project that the training data was built correctly.

9.6 A ceiling worth explaining

The citation rate of 0.5102 looks poor until one notices that it cannot exceed 0.5918 on this evaluation set: 60 of the 147 items are abstention cases, and an abstention has nothing to cite. 0.5102 means 75 of the 87 answerable items carried a citation — 86% of what is reachable.

9.7 Every arm over-fits, and the scored adapter is not the best one

Validation loss bottoms at step 50 of 150 for both Llama arms and step 75 for Mistral, then rises for the remainder of training. The adapters that were scored are the *final* step, because that is what the training framework's `save_model()` writes. The better checkpoint exists and was not the one measured.

This is recorded rather than quietly fixed because it bounds every fine-tuning number in this report from below: the arms as scored are worse than the arms as they could have been. A note in the log from 19 September — "the lowest-loss checkpoint is not the better model" — records the counter-argument, which is that validation loss and the behavioural metrics disagree about which checkpoint is best, and the behavioural metrics are what the project cares about. The question was left open rather than resolved.

9.8 Three days of infrastructure failure, recorded as failure

Arm C — Llama-3B with QLoRA — took three attempts over two days, and the log keeps all three. The first exceeded the 16 GB VRAM ceiling, which was itself the surprising part: QLoRA is supposed to use *less* memory, and it does for the 7B model. The second attempt kept the frozen output head out of fp32 and was recorded as a resolution; the next day it was corrected to "insufficient, not a resolution", because the run failed again. The third attempt succeeded and produced the numbers above.

There is also a correction in the log worth reproducing for anyone reproducing this work: `--resume` does not function in this environment and never did. A multi-hour training run that appears to resume from a checkpoint is, in this setup, silently starting over.

10 Compression for CPU

21 SEPTEMBER 2026

> The serving constraint is a CPU. The tuned adapters were merged into their bases, converted to GGUF, and the quantization ladder walked from Q8 downward — against a quality gate chosen before any build was measured.

10.1 The measurement problem, solved first

The obvious approach is to quantize, re-score on the behavioural evaluation, and keep the smallest build that does not lose quality. This does not work, and the reason is worth understanding: the 87-item answerable set has a noise floor of roughly 0.03 in grounded F1, and the quality difference between Q8 and Q5 is far smaller than that. The evaluation cannot resolve the thing being decided. It would return differences, those differences would be noise, and a build would be chosen by coin flip while appearing to be chosen by measurement.

The gate used instead is **per-token KL divergence** against the unquantized model's output distribution. This measures the damage directly rather than inferring it from downstream answer scores, and it is sensitive enough to separate adjacent rungs of the ladder.

10.2 A rule fixed before the results

The selection rule was pre-registered: **quality first, then size, then speed**. This matters because by the time results arrive there is always a build one would prefer for other reasons, and a rule written afterwards bends to accommodate it. Writing it down first is the only thing that makes "chosen by measurement" a true statement rather than a decoration.

Three reference levels were established so that different error sources stay separable, and the log records three corrections that had to be made to the measurement procedure itself before the ladder could be trusted.

10.3 What shipped

	VJLM-Llama-3B	VJLM-Mistral-7B
Base	`Llama-3.2-3B-Instruct` @ `0cb88a4f`	`Mistral-7B-Instruct-v0.3` @ `c170c708`
Adaptation	LoRA r=16, bf16 base	QLoRA r=16, 4-bit NF4 base
Adapter size	97 MB	168 MB
grounded_f1 before quantization	0.7567	0.7654
Build selected	**Q6_K + VJLM imatrix**	**Q5_K_M**
Deployed size	2.46 GB — 2.43× smaller	4.78 GB — 2.82× smaller
KL divergence	0.0097	0.0077
Change in grounded F1	+0.0000	+0.0048
CPU throughput	14.70 tok/s — ~2.4 s/answer	8.07 tok/s — ~4.3 s/answer

The Llama build is what VJLM serves. Its grounded F1 moved by exactly zero, so a 2.43-fold size reduction was obtained for no measurable quality cost.

10.4 The two models needed different compression

Mistral tolerated Q5_K_M; Llama required Q6_K. This is the substantive finding of the phase rather than an inconvenience in reporting it. Had a single quantization level been chosen for both — which is the normal practice — one of the two would have been mis-served: either Llama would have been damaged at Q5, or Mistral would have carried 20% more weight than it needed. Compression level is a property of the model, not of the project.

10.5 Q4_K_M was built and rejected

It was attractive: smaller and faster than anything else on the ladder. It is also the only build in the ladder that **breaks abstention** — it answers two questions it should refuse, reintroducing exactly the failure that fine-tuning had eliminated.

The pre-registered rule put quality first, so Q4_K_M was dropped. This is the one place in the project where the rule did real work: without it, the argument "two questions out of 147 is within noise, and the file is 30% smaller" would have been difficult to refuse, and the shipped system would silently invent regulations in a category it had been specifically trained not to.

10.6 The imatrix, and keeping the benchmark out of it

An importance matrix lets the quantizer preserve precision where it matters for a specific domain. VJLM's was computed from a 518,000-token corpus: 61% sampled from the retrieval passages, which dominate what the model actually reads at inference time, and 39% from the training set rendered in the model's own chat template.

Validation and test splits are excluded by construction. Calibrating on held-out text would leak the benchmark into the artifact being benchmarked — the compression would be tuned on the very examples later used to prove it was lossless. This is a small detail that is easy to get wrong and impossible to detect afterwards.

The compressed builds are published on Hugging Face rather than in the repository: 7.25 GB of quantized weights does not belong in version control, while the adapters, every script and every recorded result do.

11 Serving: authentication, prompt, interface

> The parts between retrieval and the student. Short, because little here was contentious — with one exception that changed how the model behaves on every answer.

11.1 Role awareness, and where access control happens

VJLM distinguishes students, faculty and guests. The important design point is that access control happens during *retrieval*, not in the prompt. By the time passages reach prompt assembly, the ones a given role may not see are already gone. A prompt instruction such as "do not reveal faculty-only material" is not a security control; it is a suggestion to a language model.

Authentication itself remains a role simulation rather than a real identity system. It is honest to call that unfinished.

11.2 The prompt, and a cue that had to be removed

The prompt is assembled in a fixed order: system instruction, role note, retrieved context, question, output rules. Citations are computed by code rather than requested from the model, because a model asked to cite its sources will produce citation-shaped text whether or not it corresponds to a real passage.

One finding here is specific enough to be useful to anyone fine-tuning a small model. The role note originally read "The user is a STUDENT. Explain plainly and practically." That phrasing was *also* the exact cue used in the role-differentiation training rows, whose answer template ends with "check this against your own regulation batch". Including the cue at inference time therefore made every student answer end with an unsupported hedge. The audience note is now phrased differently, and the hedging stopped. A fine-tuned model pattern-matches its own training cues more literally than one expects.

11.3 Refusing when the answer had in fact been retrieved

On 12 October a failure class was addressed that is the mirror image of hallucination: the system retrieved the correct passage, placed it in the prompt, and then refused to answer. The abstention behaviour that fine-tuning installed so successfully was firing even when the context was sufficient.

The fix is a rescue path: when retrieval has high confidence that the retrieved passages contain the answer, the refusal is overridden and the model is asked again. It is recorded as a decision rather than a tweak because it deliberately weakens a safety behaviour, and that trade deserves to be visible. The same day's work also recovered query terms that the tokenizer had been dropping — the "O" in "O grade", the "SEE" in "CIE and SEE" — which is a small thing that mattered a great deal for questions about grades.

12 October: the honest re-measurement

12–22 OCTOBER 2026

> Three weeks spent discovering that the retrieval system had been optimised against a benchmark that could not measure it, and rebuilding the measurement before touching the system again.

12.1 A question set written by hand

Ninety-eight questions were written as a student would actually ask them. Sixty-two came first, in three splits, and were used while tuning retrieval. Thirty-six more — `n01` to `n36` — were written afterwards and deliberately never tuned against, so that the project has at least one number that cannot have been fitted.

The difference between the two groups is the most sobering figure in the project. Current Hit@5 is 86% overall and **72% on the never-tuned thirty-six**. Every number reported on the full set is optimistic by roughly fourteen points, and that is what tuning on a question set does even when one is trying to avoid it.

12.2 The semantic arm promoted from re-ranker to retriever

Until 18 October the candidate pool came from BM25 and the semantic score only re-ordered it. The defect in that design is structural: a passage BM25 never retrieves cannot be rescued by any weighting, however good the embedding.

Measured: of 25 misses, **14 were passages BM25 ranked between 300th and 9,000th** — unreachable by reweighting — while the embedding placed them at ranks 2, 16, 22 and 37. The reverse also occurs, which is what settles the design: the question about leftover food at the hostels has its answer at BM25 rank 7 and cosine rank 908. Neither arm can be trusted to own the pool.

So the candidate set became the *union* of both rankings, with every candidate scored by all three signals. Hit@5 moved from 74% to 83%. Deepening the semantic pool from 300 to 1,200 then moved Hit@100 from 95% to 98%, for about 60 milliseconds a query, because gold passages sit as deep as cosine rank 1,156. Beyond 1,200 nothing further is reached — 4,000 measured identical.

12.3 A fourteen-point leak between ranking and the model

This is the largest single defect found in the project, and it had been invisible because the instrument that would have caught it was measuring one step too early.

Hit@5 was 83%. But Hit@5 asks whether the answer appears in the five *passages selected*. The model does not see whole passages; it sees each passage trimmed to fit a character budget, with the sentences that best match the question kept. Measuring the question the model actually faces — is the answer in the prompt text? — gave **69%**.

Fourteen points of correctly retrieved answers were being discarded after ranking. Two causes, in series:

1. The trimming function dropped any sentence scoring below half the best-scoring sentence. But the answer is typically the sentence the question's words are *absent* from — "in which area of Hyderabad

is the campus" is answered by a place name the question cannot contain — so the answer scored low and was filtered out of a passage retrieval had correctly chosen.

2. That filter ran *before* the character budget, which made the budget inert: every surviving sentence already fit, so raising the budget changed nothing. The prompt sat at 491 tokens of a 4,096-token context. The system was using twelve per cent of its context window.

Low-scoring sentences now go last rather than away, and the budget was raised from 3,250 to 5,000 characters. Answer-in-prompt went 69% → 79% → **86%**, which is exactly Hit@5. The leak is closed; the prompt now loses nothing that ranking found. Beyond 5,000 characters nothing changes, so that is the smallest budget that leaks nothing.

12.4 A configuration split that invalidated the numbers

Found while explaining the retrieval formula, which is a good argument for having to explain one's own system. The application defaulted to a semantic weight of **0.5**. All four evaluation harnesses hard-coded **1.5**. Nothing set the environment variable that would have reconciled them.

So every retrieval figure measured in October described a configuration the application did not serve. Both now read a single default, and the harnesses override it only when explicitly asked. It is a two-line fix for a defect that silently detached three weeks of measurement from the deployed system.

12.5 What the weights are worth, swept properly

With the measurement trustworthy, the weights were searched rather than guessed: 36 configurations over part-of-speech, synonym gate, slot cap and synonym weight, each scored on the tuning split and reported separately on the held-out and never-tuned splits.

Change	Hit@5	Where the gain landed
Starting point	83%	—
Synonym weight 0.5 → 1.0	85%	Entirely on the never-tuned split, 23 → 25 of 36
+ IDF gate on WordNet expansion	86%	Never-tuned split 25 → 26 of 36
Part of speech, slot cap, gate ratio 1.3	no change	Twelve configurations tie at 86%

Twelve settings tie at the top and none exceeds it. The weight-tuning space is exhausted; further gains must come from somewhere else.

13 The three-signal retrieval metric

> The measure that ships. It is the supervisor's coverage ladder with a third signal added, and set the semantic weight to zero and the original measure is recovered exactly.

13.1 The formula

$$R(q, c) = 1.0\,K + 1.0\,W + 1.5\,S + 0.2\,B$$

Each component lands in [0, 1], which is what makes the weights comparable rather than arbitrary.

K — direct keyword matching

$$K = \frac{\Sigma_i \, \mathrm{idf}_i \cdot \mathrm{E}_i}{\Sigma_i \, \mathrm{idf}_i} \qquad \mathrm{E}_i = 1 \text{ if term } i \text{ or an inflection of it is present}$$

W — WordNet and domain synonym matching

$$W = \frac{\Sigma_i \, \mathrm{idf}_i \cdot (1 - \mathrm{E}_i) \cdot \mathrm{N}_i}{\Sigma_i \, \mathrm{idf}_i} \qquad \mathrm{N}_i = 1 \text{ if a synonym is present}$$

The $(1 - E_i)$ factor matters: a term already matched exactly is never counted a second time as a synonym match. In the implementation this is not a multiplication but an `if/elif` — each term's IDF mass is assigned to *K* or to *W*, never to both. Splitting the original coverage computation into two separately weightable halves is what makes the three-signal formula possible at all.

S — semantic matching

$$S = \frac{\cos(q, c) - \min_{\text{candidates}}}{\max_{\text{candidates}} - \min_{\text{candidates}}}$$

Normalised across the candidate pool, not the corpus. A corpus-wide scale was tried and rejected: it pushes every candidate near 1.0 on a query whose topic is common and near 0 on a rare one, at which point the weights stop meaning anything.

B — tie-break

A small normalised BM25 term, weighted 0.2, which separates passages covering identical terms. It is not a fourth signal; it breaks ties.

13.2 A worked example

Question: *"When do first year classes start?"* Terms `first`, `year`, `classes`, `start`. Suppose the IDF mass divides 0.1 / 0.1 / 0.3 / 0.5 — `start` is the rarest and so the most discriminating.

The academic calendar passage reads *"Commencement of Classes | 10 August 2026"*:

Term	Present?	Contributes to	Value
`classes`	Yes, literally	*K*	0.3
`start`	No — but `commencement` is	*W*	0.5
`first`, `year`	No	—	0

$$R = 1.0(0.3) + 1.0(0.5) + 1.5(0.82) = \mathbf{2.03}$$

Without the synonym arm this passage scores 0.3 on words and loses to any page that happens to repeat "first year classes" without answering the question. That bridge is the reason the calendar question works at all, and it is the clearest illustration of why the supervisor's half-credit tier belongs in the measure.

13.3 Retrieval in four steps

1. **Gather candidates.** BM25 returns its top 60; the embedding returns its top 1,200. The candidate pool is their union, so a passage either arm can find is scored by all three signals.
2. **Score every candidate** on *K*, *W* and *S*, each on a 0–1 scale.
3. **Fold in keyword evidence and priors.** Section-heading matches, identifiers the tokenizer drops, and terms appearing only in a filename (`II-B.Tech-CSBS-R25.pdf` — its tables never repeat the branch) are added to the *keyword* arm. Wrong-branch passages are multiplied by 0.35; table-of-contents pages by 0.4.
4. **Take the top five, with a diversity rule.** Walk the ranking and skip a passage whose document has already contributed its quota — except for rules, course and regulation questions, where one document legitimately owns the whole answer and the cap is lifted.

13.4 Why the weights are not normalised to sum to one

A sum-to-one presentation — 0.45 / 0.25 / 0.30 — is more readable and was tested. Two things are true about it. First, normalising weights cannot change any ranking: only the *ratios* matter, so dividing all three by their sum is purely cosmetic. Second, those particular ratios are not a normalisation of the shipped weights — they cut semantic from 1.5× keyword to 0.67×, a factor of 2.3 — and they measure **3 points worse** (80% against 86%).

The shipped weights also depart from coverage's original 1 / 0.5 / 0 ladder in one respect: the synonym weight is 1.0, equal to the keyword weight, so a synonym match counts as much as a literal one. Coverage uses 0.5 to prevent a synonym from outranking a literal hit. In the hybrid the semantic arm

breaks those ties, so the caution costs more than it saves — worth 2 points, entirely on never-tuned questions. Coverage mode itself is unchanged and still selectable.

14 The WordNet question, answered with numbers

> WordNet is one of the three signals the project was asked to build. It is implemented, live, and carries a non-zero weight. Measured in isolation, its marginal contribution on this question set is zero. Both halves of that sentence are true and both belong in the report.

14.1 What is actually implemented

Synonym expansion runs through `nltk.corpus.wordnet`: real synsets, part-of-speech restricted, sense-ordered, with derivational forms included and multi-word lemmas dropped because the lexical index holds single tokens. Expansions are gated on the indexed vocabulary — a synonym that appears in no VNRVJIET document is discarded before it can add noise, since it could not match anything anyway. Alongside WordNet sits a small hand-curated domain map for vocabulary WordNet does not have: `commencement`, `condonation`, `revaluation`, `backlog`, `sanctioned intake`.

14.2 The isolation measurement

The two synonym sources were switched off independently, everything else held fixed:

Configuration	Hit@5	Marginal contribution
Both sources active — what ships	84 / 98	—
WordNet disabled	84 / 98	**+0 points**
Domain map disabled	80 / 98	+4 points
No synonym matching at all	80 / 98	—

WordNet contributes nothing measurable. The hand-built domain map contributes four questions. And WordNet *alone*, with the domain map disabled, scores identically to no synonym matching at all — so this is not a case of two sources overlapping and splitting the credit.

The explanation is straightforward once seen: every expansion WordNet gets right on this corpus is already in the domain map. `start` → `commencement` and `rule` → `regulation` are both there, hand-written, because they were needed before WordNet was working well enough to supply them.

14.3 It was improved rather than left as dead weight

The arm had been restricted to **nouns only**, and that restriction was based on real evidence from 26 September: unrestricted expansion was overwhelmingly noise, and verbs were the worst of it — `attend` → `serve`, `miss` → `girl`, `do` → `bash`. WordNet ranks senses over general English, not over academic regulations.

But the questions that need a synonym need a *verb* one. "When do classes **start**" needs `commencement`; "can I get my marks **rechecked**" needs `revaluation`; "exams for **failed** subjects" needs `supplementary`. And `start` expanded to nothing at all, despite WordNet's `start.v.01` holding exactly the lemmas `begin` and `commence` — because synsets were pooled across parts of speech, letting noun senses crowd out the verb reading entirely.

The fix is a sense filter rather than a blanket ban, and it comes from noticing a pattern in which senses were wrong. Every bad expansion — `girl`, `serve`, `nine`, `place` — is a **common** word. Every useful one — `commencement`, `revaluation`, `condonation`, `supplementary` — is **rare**. Rarity in this corpus is already measured, as IDF:

$$\text{keep synonym } s \text{ for term } t \iff \text{idf}(s) \geq \rho \cdot \text{idf}(t)$$

Accept a synonym only if it is at least as discriminating, *in this corpus*, as the word it replaces. A wrong sense is generic by nature and fails the gate; a rare technical term passes. Verified against the real IDF map: `seat` → `place` and `apply` → `use` are now dropped, `start` → `commencement` and `rule` → `regulation` kept. The gate is worth one point, and it lets verbs back in safely — at $\rho = 1.0$, with $\rho = 1.3$ measuring identically.

14.4 How to present this honestly

The temptation is to report that the project implements a three-signal hybrid including WordNet-based synonym matching, which is true, and stop there. The better claim is the measured one: *each arm's marginal contribution was isolated, and here is what each is worth.* Keyword matching carries 71 points of the 86. Semantic matching adds 11. Synonym matching adds 4, all of it from the domain map, none from WordNet.

That is a stronger result than an unmeasured formula, because it is the kind of statement that survives questioning. It also identifies exactly what would make WordNet earn its place: a corpus whose vocabulary gaps are not already hand-covered, or a domain map small enough that WordNet has room to contribute. Neither is true here, and saying so is more useful than implying otherwise.

15 Where the system stands

22 OCTOBER 2026

Measure	Value	Note
Hit@5, all 98 questions	**86% — 84/98**	Answer present in the five passages shown
Hit@5, 36 never-tuned questions	72% — 26/36	The unbiased figure
Answer present in candidate pool	99%	Nothing is unreachable
Answer survives into prompt text	86%	Equals Hit@5 — no loss after ranking
Retrieval latency	~0.2 s / query	Union pool, 1,200 deep
Generation	14.70 tok/s	Q6_K, CPU, 20 threads
Corpus	104,965 passages	3,297 documents, 0 parse failures
Answer-level accuracy	~64%, stale	Last measured before the 22 October fixes

15.1 What moved the number on 22 October

Three changes took Hit@5 from 83% to 86%, and closed a leak that had been costing far more than that:

1. **The prompt-assembly leak** (section 12.3). Answer-in-prompt went from 69% to 86%. This is the largest effect of the day by a wide margin: it did not raise Hit@5, it made Hit@5 *true*.
2. **Synonym weight and the IDF gate** (sections 12.5, 14.3). +3 points, all on never-tuned questions.
3. **The configuration split** (section 12.4). No effect on the measured number; it made the measured number apply to the deployed system.

Two chunker defects were also fixed (section 6.2). They affect future extractions and do not change today's figures.

15.2 The fourteen questions that still fail

All fourteen have their answer in the candidate pool. Where it sits in the final ranking determines what would be needed to recover it:

Gold rank	Questions	What would recover them
6–8	4	A modest ordering gain — or showing 8 passages instead of 5
13–19	5	Finer chunking; score dilution is the suspected cause
25–30	3	Finer chunking
Below 200	2	Structural — one page is a single undivided blob

Four questions short of 90%, nine short of 95%. The four nearest all sit at ranks 6 to 8, which is tantalisingly close and also exactly the band that eleven separate re-ranking attempts have failed to reach.

16 What is not done

> The numbers in this report are worth only what their caveats allow, so the caveats are stated at the same volume.

16.1 Answer accuracy is not Hit@5, and it is much lower

Hit@5 says the answer was retrieved. Whether the model then wrote it down correctly is a separate question with a worse answer: **57% by strict pattern match** over the 98 questions, and roughly **64%** after hand-grading all 42 failures (7 were correct answers the pattern scored wrong, 4 were genuinely ambiguous, 31 were wrong). That measurement predates the 22 October fixes and has not been repeated, because it requires the model server and about thirty minutes. It should be the next thing run.

Of the 42 failures, the classes were: 17 retrieval misses, 8 table-reading failures where the model could not extract a value from a flattened table, and 8 abstentions where the answer sat in the prompt but the model refused anyway.

16.2 The accuracy targets are not met

The target was 90%, then 95%. Hit@5 is 86%: 90% needs 88 of 98, 95% needs 93. Because the answer is in the pool for 99% of questions, both remain reachable in principle. Neither is reachable by weight tuning, which is exhausted.

16.3 Eleven ordering ideas measured and rejected

Recorded with the number that killed each, so that none is attempted twice:

Idea	Result
Cross-encoder re-ranking	73%, and 664 ms/query
Best-sentence re-ranking of the leaders	81%
Section-overview / hub-page bonus	83%, Hit@3 worse
IDF raised to a power	flat at 1.5, 2, 3
Reciprocal-rank fusion of the arms	72%
Enumeration / repetition bonus	83%, then 81%
Additional domain synonyms	flat
Larger dense pool under the old design	flat
Prompt re-ordering by best sentence	identical, 23/26
Per-document caps	79% against 83%
Weights normalised to sum to 1	80%

The reason this list is so uniformly negative is diagnostic rather than bad luck. In 6 of 12 reachable misses, the passage that wins covers *more* of the question than the passage holding the answer — often

all of it. "In which area of Hyderabad is the campus" is answered on one page scoring 0.64 and loses to another scoring 1.00, because the answer is a place name the question cannot contain. No reweighting of query terms fixes that; the winner really is a better match for the words asked. The problem is not the ordering function.

16.4 The most likely remaining lever is chunk granularity

Gold passages are 1,100 characters and the answer is one sentence inside them, so every signal scores the answer against roughly 900 characters of unrelated text in the same passage. The chunker packs multiple paragraphs up to that cap by design (section 6.1).

An earlier structural-chunking experiment at finer granularity measured **better at Hit@5 — 74% against 72%** — but worse at greater depth, and was rejected on the deeper numbers. For a system that only ever shows five passages, the deeper numbers do not matter, so that rejection used the wrong criterion. It is the only lever that has ever improved Hit@5 specifically.

It requires a re-extraction and a re-index — hours of unattended compute — and it remains a genuine bet rather than a certainty.

16.5 Known measurement weaknesses

- 98 questions is a small set, and 62 of them were written while retrieval was being tuned. Only the 36 never-tuned questions give an unbiased reading, and there the figure is 72%, not 86%.
- The gold patterns test whether the answer *text* was retrieved, not whether a reader would judge the generated answer correct. Seven of the 42 answer-level failures were correct answers scored wrong by their pattern.
- One extraction defect is unfixed: OCR misreads superscript ordinals in scanned PDFs, so `06th January` becomes `06m January`. A reader sees past it; a keyword match cannot.
- Authentication is a role simulation, not an identity system.
- The fine-tuned adapters that were scored are the final training step, not the best checkpoint, so every fine-tuning figure is a lower bound (section 9.7).

17 What the project taught about method

> The technical content of this report is a retrieval system and a compressed model. Its more transferable content is four occasions on which a measurement was confidently wrong, and what each one had in common.

17.1 Six measurements that agreed, and all measured nothing

The per-document cap — how many passages a single document may contribute to the five shown — was measured six times with different settings. All six agreed. The agreement was meaningless.

The harness used retrieves at depth 100, and the caps scale with the number of passages requested, so at depth 100 they never bind. Six settings produced identical results because not one of them exercised the thing being varied. The error surfaced only when a question whose answer sat at rank 5 nonetheless produced an abstention — a result impossible under any of the six readings. Re-measured with a harness that retrieves the way the application does, the capped and uncapped configurations differ by four points.

The lesson is narrow and worth stating precisely: **agreement between measurements is not evidence of correctness.** Six consistent results are also what a broken instrument produces.

17.2 The same error class, four times

Four defects in this report share a structure: the system produced output that looked correct while operating on the wrong thing.

Defect	What it reported	What was true
Stale dense cache	Passages retrieved	Embeddings described the old text
Vector misalignment	Plausible semantic scores	Wrong passages, consistently
Image-only PDFs	672 of 672 processed	25 contributed nothing
Cap measured at depth 100	Six consistent results	The cap never engaged
Prompt trimming filter	Hit@5 83%	The model saw the answer 69% of the time

None raised an exception. Each would pass a test asserting that results come back. What catches this class of defect is not more tests of that kind but a different question: *was the thing I am measuring actually exercised?*

17.3 Pre-registration did real work exactly once

The quantization selection rule — quality, then size, then speed — was written before any build was measured. It earned its keep when Q4_K_M turned out smaller and faster and broke abstention on two of 147 items. Written afterwards, the rule would almost certainly have accommodated "two items is within noise".

The inverse also appears. The retrieval benchmark was built before components were compared, which the project's own phase gates require — but building it first did not make it valid, and its invalidity cost

three weeks of work optimising against a signal weighting it had wrongly recommended. Order of operations is not a substitute for construct validity.

17.4 Recording what failed

The decision log keeps superseded entries, failed attempts and corrections rather than editing them out: three hardware decisions superseded when the machine changed, two failed fixes for the same VRAM ceiling, a resolution downgraded the next day to "insufficient", two design-document predictions refuted by their own experiment, and an explicit correction retracting an earlier claim about per-document caps.

This has been useful in a specific, repeated way. Twice, an idea that seemed new turned out to be recorded as already measured and rejected, with the number attached. The log did not merely document the work; it prevented a measurable amount of it from being done twice.

Appendix A Full dated timeline

Dates from the repository's commit history (github.com/nvaishnavisriyaa/VJLM) and the dated entries of `DECISIONS.md`.

Date	Commit / record	Event
16 Sep	`6c7c22b`	Phase 0 — repository foundation
16 Sep	`33bcbd6`	Ingestion dependency policy (INGEST-001); Colab operating arrangement
16 Sep	`ab8ddfd`	Phase 1 — site reconnaissance; the docs-subdomain blocker
16 Sep	`e4a02d1`	Phase 2 — fine-tune vs RAG classification, 14 categories
16 Sep	`d7461a9`	Working pipeline: discovery, fetch, extract, index, serving app
16 Sep	`0408f2d`	Layer 1 enrichment; 39× local inference regression fixed
16 Sep	`86fc35f`	Self-contained Colab pipeline; generation evaluation harness
17 Sep	`d085088`	HANDOFF for the move to a GPU machine; platform binaries untracked
17 Sep	`605dd33`	Master plan, source document and architecture figures
17 Sep	`1f9835b`	**Train/test leakage fixed** in the fine-tuning dataset
17 Sep	`d304482`	Self-maintaining pipeline: app-driven refresh, headless rendering
17 Sep	`d6d8e40`	First full end-to-end run; first dense index
17 Sep	`ceefbd7`	**Stale-vector bug** in the dense cache fixed; rendering hardened
17 Sep	`478183a`	Freshness gated on the deploy signal, with a floor
18 Sep	`9929c78`	Phase 8 — Mistral-7B QLoRA trained and scored; long runs made survivable
18 Sep	`acfcc1f`	Arm A scored — **the 3B model matches the 7B**
18 Sep	SLM-001, FT-002/003	Base model and method decided; three earlier entries superseded
19 Sep	`8ba385e`	Arm C exceeds 16 GB — QLoRA costs more VRAM than LoRA here
19 Sep	`4958b97`	First fix attempt: keep the frozen head out of fp32

19 Sep	`33e389c`	FT-005 corrected — that fix was insufficient, not a resolution
19 Sep	`8158aa0`	**Phase 8 complete** — LoRA vs QLoRA is a wash on quality
19 Sep	`468581f`	Recorded: the lowest-loss checkpoint is not the better model
21 Sep	`1874e4c`	Phase 9 — compress for CPU; adapters published
21 Sep	`27bc9b7`	**Phase 9 complete** — the two models needed different compression
21 Sep	`ff1540a`	GGUF builds ship from Hugging Face, not GitHub
26 Sep	RAG-SIM-001	Eight similarity modes on 90 questions; coverage selected; two design predictions refuted
26 Sep	paraphrase_probe	Unrestricted WordNet expansion shown to be noise; nouns-only adopted
30 Sep	CORPUS-001	**Corpus holds almost no student-policy content**; classifier at fault
30 Sep	INGEST-003	25 of 672 PDFs have no text layer; OCR added; space-dropping repaired
30 Sep	RAG-IDX-001	**Dense vectors misaligned with the lexical index**
12 Oct	`8e50fab`	Dropped query terms recovered; answer instead of refuse; coverage as default
17 Oct	—	Corpus rebuilt: 104,965 passages, 3,297 documents, 0 parse failures
18 Oct	RAG-EVAL-004/005	Three-signal fusion; union pool; pool deepened to 1,200; branch mismatch penalised; six variants rejected
22 Oct	—	**14-point prompt leak fixed**; IDF-gated WordNet; 36-config sweep; corpus audit; chunker defects fixed

Appendix B Every rejected experiment

> Each with the measurement that ended it. Recorded so that no idea here is tried a second time, and so that the shipped configuration can be read as the survivor of a search rather than a guess.

Retrieval ordering

Idea	Measured	Date
Cross-encoder re-ranking of the top candidates	Hit@5 73%; 664 ms/query	18 Oct
Best-sentence re-ranking of the leaders	81% at weight 1.0; 79% at 2.0	18 Oct
Section-overview / hub-page bonus	83% unchanged; Hit@3 77% → 74%	18 Oct
IDF raised to a power	flat at 1.5, 2 and 3	18 Oct
Reciprocal-rank fusion instead of a weighted sum	72%	18 Oct
Enumeration / repetition bonus for list questions	83%, then 81%	18 Oct
Additional hand-written domain synonyms	flat	18 Oct
Larger dense pool under the BM25-only pool design	flat	18 Oct
Ordering the chosen passages by best sentence	identical — 23/26, same failures	18 Oct
Per-document caps on the five shown	79% against 83% uncapped	22 Oct
Weights normalised to sum to 1 (0.45/0.25/0.30)	80% against 86%	22 Oct
WordNet part-of-speech widening; slot cap 6; gate 1.3	all identical to the shipped setting	22 Oct

Passage count and prompt

Idea	Measured	Date
10 passages instead of 5	end-to-end 22/26 against 23/26; +36% latency	18 Oct
20 passages instead of 5	17/26	18 Oct
Prompt budget beyond 5,000 characters	6,500 and 9,000 measured identical	22 Oct

Embedding and chunking

Idea	Measured	Date
snowflake-arctic-embed-xs in place of bge-small	76% against 77%	18 Oct
Corpus-wide normalisation of semantic scores	rejected — candidates collapse near 1.0 or 0	18 Oct
Structural chunking (v2)	@5 74% vs 72%, but @20 80% vs 84%	18 Oct

Model and compression

Idea	Measured	Date
Mistral-7B as the deployed model	0.7654 vs 0.7567, at 3× training cost and half the speed	18 Sep
QLoRA for the 3B deployment arm	quality identical; 42% slower to train	19 Sep
Q4_K_M quantization	**breaks abstention — answers 2 it must refuse**	21 Sep
Content-hash change detection	every source changes on every deploy	17 Sep

Appendix C Repository map

Path	Contents
`configs/`	Versioned configuration — no hard-coded paths or model names
`data_catalog/`	Source classification: fine-tune / RAG / both / neither
`ingestion/`	Registry, crawler, raw snapshots, per-source sync state
`preprocessing/`	Two pipelines: RAG chunking, and fine-tuning dataset building
`rag/`	Embedding, index, retrieval, scoring, prompt assembly
`evaluation/`	Question sets, harnesses, ablations, diagnostics
`training/`	Datasets, notebooks, adapters
`quantization/`	GGUF conversion and quality / speed / RAM measurements
`auth/`	Authentication and RBAC policy
`app/`	Backend API and the student / faculty interface
`experiments/`	Reproducible runs and raw results
`docs/`	Hardware profile, runbooks, phase deliverables, this report
`DECISIONS.md`	~1,600 lines: every decision with the measurement behind it

Key modules

File	Role
`rag/retrieve.py`	The three-signal scorer, candidate union, caps, prompt assembly
`rag/coverage.py`	The supervisor's IDF-weighted term ladder
`rag/synonyms.py`	WordNet expansion, the IDF gate, the domain map
`rag/scope.py`	Role and branch priors, query parsing, document profiles
`rag/focus.py`	Per-passage trimming — the 14-point leak lived here
`preprocessing/extract.py`	Parsing, OCR fallback, chunking
`preprocessing/enrich.py`	Metadata, redaction, deduplication

Evaluation harnesses written during this work

Harness	Question it answers
`hit_at_prompt.py`	Hit@5 with the per-document caps actually binding
`hit_in_prompt_text.py`	Does the answer survive trimming into the prompt?
`ablation.py`	What is each of the three signals worth?
`sweep_wordnet.py`	Part of speech × IDF gate × slot cap × synonym weight
`audit_corpus.py`	Is the knowledge base at fault, or is retrieval?
`probe_wordnet.py`	What does the IDF gate keep and drop, on the real corpus?
`why_miss.py`	Which arm is responsible for each miss?
`diagnose_misses.py`	Recall problem or ranking problem?
`depth_curve.py`	Hit@k across depth
`screen_embedders.py`	Candidate embedding models, without re-embedding the corpus

VJLM — Vignana Jyothi Language Model. Report compiled 22 October 2026 from the repository's commit history, decision log and recorded experiment results. All figures in this document are measurements taken on the system described; where a figure is stale or uncertain, the text says so.